Distributed Knowledge-Enhanced Multiagent Reinforcement Learning for Internet of Drones

Ye Wang, *Student Member, IEEE*, Jingjing Wang, *Senior Member, IEEE*, Ruijie Zhu, *Member, IEEE*, Ziyang Wang, *Member, IEEE*, Pujie Xin, *Member, IEEE*, and Peng Pan, *Member, IEEE*

***Abstract*—With the growing demand for low-altitude transportation, the application of the Internet of Drones (IoD) in urban logistics has become increasingly significant. However, the complex obstacles present in urban environments, such as tall buildings and no-fly zones, pose numerous challenges for the IoD, including low data-driven efficiency and difficulties in representing implicit knowledge. To address these challenges, this article proposes an IoD swarm collaborative scheduling method based on distributed knowledge-enhanced multiagent reinforcement learning (DKEMARL). The method leverages prior environmental knowledge to design a knowledge embedding and expansion module, which provides comprehensive and detailed environmental observation data during training and introduces a reward mechanism that balances task timeliness with flight safety. Within a centralized training and decentralized execution framework, the multiagent deep deterministic policy gradient (MADDPG) approach is employed to facilitate efficient cooperation among the IoD. Specifically, we develop a scenario model for low-altitude transportation tasks involving an IoD swarm, considering factors such as task timeliness, flight distance cost, and safety constraints, and propose an optimization objective to balance timely task completion with flight safety. Simulation results demonstrate that the proposed DKEMARL algorithm can significantly enhance task completion efficiency compared to baseline methods that do not incorporate knowledge enhancement.**

***Index Terms*—Collaborative scheduling, data-driven, drones, knowledge-driven, multiagent reinforcement learning (MARL).**

Received 13 May 2025; accepted 17 June 2025. Date of publication 23 June 2025; date of current version 24 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62371424, Grant 62222101, and Grant U24A20213; and in part by the Beijing Natural Science Foundation under Grant L232043 and Grant L222039. *(Corresponding author: Ruijie Zhu.)*

Ye Wang is with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China, and also with the School of Computer Science and Artificial Intelligence, Zhengzhou University, Zhengzhou 450001, China (e-mail: yewangcs@gmail.com).

Jingjing Wang is with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China, and also with Hangzhou Innovation Institute, Beihang University, Hangzhou 310051, China (e-mail: drwangjj@buaa.edu.cn).

Ruijie Zhu is with the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with the School of Computer Science and Artificial Intelligence, Zhengzhou University, Zhengzhou 450001, China (e-mail: zhuruijie@bupt.edu.cn).

Ziyang Wang is with the School of Electronic Information Engineering, Beihang University, Beijing 100191, China (e-mail: wangziyang1@buaa.edu.cn).

Pujie Xin and Peng Pan are with the School of Communications Engineering, Hangzhou Dianzi University, Hangzhou 310018, China (e-mail: 43096@hdu.edu.cn; panpeng@hdu.edu.cn).

Digital Object Identifier 10.1109/JIOT.2025.3582058

I. Introduction

With the continuous acceleration of urbanization and the rapid growth in logistics demand, traditional ground transportation systems have progressively revealed their inefficiencies and lack of timeliness. Against this background, the Internet of Drones (IoD) leverages its exceptional maneuverability, rapid deployment capabilities, and extensive coverage to enable interconnected collaboration via wireless networks [1]. This technology not only overcomes the limitations imposed by urban traffic congestion to facilitate efficient delivery, but also demonstrates significant application potential in critical scenarios such as emergency medical services, express delivery, and urban material dispatch [2].

However, urban airspace is characterized by a complex and dynamic environment. The prevalence of high-rise buildings, scattered no-fly zones, and unexpected obstacles imposes increased uncertainties and safety risks on IoD swarms during path planning and task allocation [3], [4]. Conventional scheduling methods relying on historical statistics or fixed heuristics often fail to leverage real-time environmental data and uncover latent situational insights, leading to locally optimal solutions that degrade safety and mission success rates. To remedy this, prioritized information bottleneck frameworks have been introduced for distributed online learning in edge video analytics, enabling drones to filter and update critical sensory data on the fly [5]. Similarly, priority-aware collaborative perception schemes allocate sensing resources according to task urgency, improving cooperative awareness in vehicle-to-everything networks [6].

As drone swarms grow in size and mission complexity, maintaining seamless, low-latency links and ensuring data security become critical prerequisites for reliable operations. To this end, reconfigurable intelligent surfaces have been integrated into drone communication systems: Huang et al. [7] proposed an RIS-assisted trajectory planning framework that exploits multithreading and federated learning to both speed up path decisions and stabilize air–ground connectivity under dense deployments. Complementing this, blockchain-based access control mechanisms have been deployed to authenticate drone identities and protect data integrity across distributed drone networks [8]. Moreover, Wang et al. [9] introduced an adaptive chained byzantine fault-tolerant consensus protocol tailored for drone ad hoc networks, further bolstering resilience against malicious or malfunctioning nodes without sacrificing throughput.

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:46:07 UTC from IEEE Xplore. Restrictions apply.

Despite these advances, existing methods often address communication performance or security in isolation and rarely leverage prior environmental knowledge to guide decision-making. Therefore, the central challenge remains: how to accurately capture and utilize dynamic environment data, extract valuable implicit knowledge, and guarantee flight safety, in order to achieve efficient coordination among IoD swarms in low-altitude transportation tasks [10].

A. Related Works

Multidrone task allocation and path planning are critical for enhancing the efficiency of urban low-altitude transportation systems. Early graph-based and mathematical optimization approaches, such as A* [11], Dijkstra [12], mixed integer programming [13], integer linear programming [14], and auction algorithms [15], provide feasible solutions in static or semidynamic settings, but lack adaptability to real-time disruptions.

Inspired by biological swarm intelligence, techniques like ant colony optimization [16], particle swarm optimization [17], genetic algorithms [18], and other evolutionary algorithms [19] introduce self-organizing rules for cooperative path planning. However, their high computational complexity and slow convergence in large-scale, high-dimensional tasks limit real-world applicability.

To overcome these limitations, deep reinforcement learning (DRL) has emerged as a promising direction. DRL-based path planning dynamically adapts drone trajectories, boosting obstacle avoidance and path efficiency. Within this paradigm, multiagent reinforcement learning (MARL) is a core enabler. Policy-based MARL methods, such as multiagent deep deterministic policy gradient (MADDPG) [20], COMA [21], MAPPO [22], and MAAC [23]—optimize joint policies via policy gradients to enhance swarm coordination in nonstationary environments. Value-based approaches—VDN [24], QMIX [25], QPLEX [26], and CollaQ [27]—decompose the global value function into local estimates, reducing search space and improving computational efficiency for collaborative tasks.

Beyond path planning, information coverage has also benefited from MARL. For example, a dense multiagent RL framework achieves significant gains in coverage rate for vehicular networks by coordinating multiple drones in complex scenes [3], [28]. These advancements lay a solid foundation for our work on efficient, safe IoD swarm coordination in dynamic urban airspaces.

B. Motivations and Contributions

To address the aforementioned problems, this article proposes a distributed knowledge-enhanced MARL (DKEMARL) algorithm. As shown in Fig. 1, the algorithm leverages environmental prior knowledge to design a knowledge embedding and extension enhancement module, which provides comprehensive and detailed environmental observations during training and introduces a reward mechanism that balances task timeliness with flight safety. Moreover, it builds on the MADDPG framework under a centralized training and decentralized execution paradigm—utilizing global information during training to optimize drone scheduling strategies and relying on observations from the IoD during execution to enable efficient task cooperation among IoD swarms. The contributions of this article are as follows.

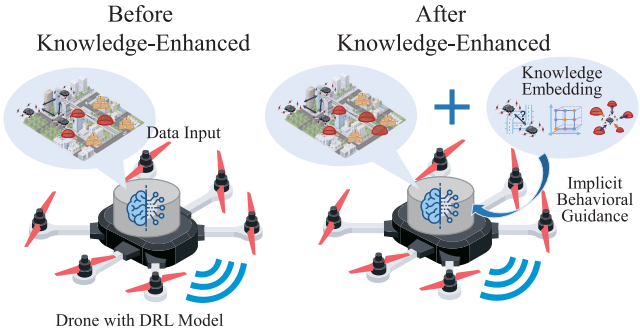

Fig. 1. Comparison before and after knowledge-enhanced.

1) This article establishes a comprehensive model for a low-altitude transportation scheduling system for IoD swarms, incorporating factors such as drone motion, task assignment, obstacle avoidance, and flight safety. By considering transportation timeliness, flight distance costs, and safety distance constraints, the system ensures effective task execution across diverse environments.
2) Addressing the challenges of low data-driven efficiency and the difficulty of representing implicit knowledge in traditional scheduling methods in complex urban environments, this article innovatively proposes the DKEMARL algorithm. It also constructs a comprehensive reward mechanism with safety constraints, thereby enhancing the robustness and safety of the low-altitude transportation scheduling system.
3) Utilizing prior data obtained from the environment, this article designs a knowledge embedding and extension enhancement module. By constructing multiple knowledge bases, including obstacle distance, target distance, recommended heading, risk assessment, and estimated collision time, the module effectively integrates drone state information with latent environmental structural data, providing comprehensive and detailed environmental observations for subsequent policy learning.

II. System Modeling and Problem Statement

In low-altitude transportation, due to obstacles such as tall buildings and no-fly zones, drones must plan avoidance trajectories to complete tasks. As shown in Fig. 2, this is the description of low-altitude transportation scenarios in the IoD. We assume that all drones fly at the same altitude and represent obstacles with circular areas, where the center and radius of the circle denote the position and size of the obstacles, respectively. Additionally, the system defines loading and unloading points, with a predefined quantity of goods at each loading point, and all goods are consistent in

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:46:07 UTC from IEEE Xplore. Restrictions apply.

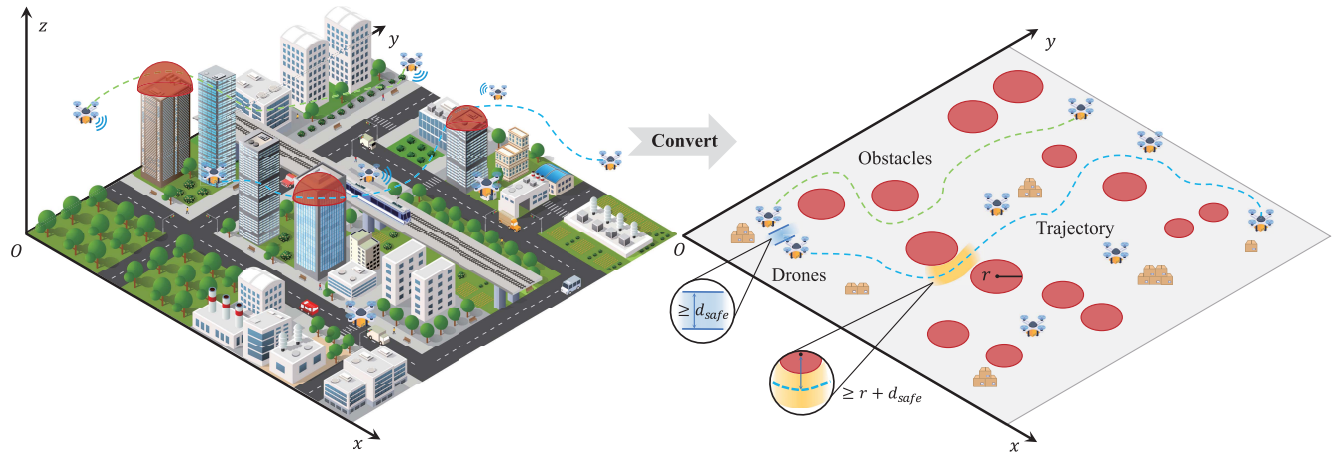

Fig. 2. Description of low-altitude transportation scenarios in the IoD.

TABLE I
LIST OF NOTATIONS FOR SYSTEM MODELING

Symbols	Meanings
$\mathbb{R}^2$	2D Euclidean space for transport
$\mathcal{P}$	Set of loading points
$p_i = (x_i, y_i)$	Coordinates of loading point i
$\mathcal{D}$	Set of delivery points
$d_j = (x_j, y_j)$	Coordinates of delivery point j
$\mathcal{O}$	Set of obstacles
$o_l = (x_l, y_l)$	Coordinates of obstacle l
r_l	Radius of obstacle l
$\mathcal{U}$	Set of drones
u_k	Drone k
$u_k(0) = (x_k^0, y_k^0)$	Initial position of drone k
$v_k(t)$	Velocity of drone k
$c_k(t)$	Cargo load of drone k
C	Drone's payload capacity
d_{safe}	Minimum safe distance
T	Total task time
x_{ijk}	Binary task assignment variable
q_{ij}	Cargo between loading and delivery points

size and weight. Therefore, when the drone swarm performs the transportation task, not only must the flight trajectories be reasonably planned, but the transportation tasks must also be effectively allocated to ensure the accurate and timely transportation of goods from loading to unloading points. Furthermore, to ensure flight safety, the system imposes strict regulations on the minimum safety distance between drones and between drones and obstacles to avoid collisions. In summary, the core scheduling objective of this article is to complete all transportation tasks based on a comprehensive consideration of total transportation time and flight distance costs. The notation used for the system modeling is presented in Table I.

A. Scenario Description and Model Construction

Assuming that all drones operate at a uniform flight altitude, we simplify the low-altitude transportation scenario to a planar system in the 2-D Euclidean space $\mathbb{R}^2$, where the problem of drones executing transportation tasks is studied. All positions are represented in a Cartesian coordinate system, and the set of loading points is defined as

$$\mathcal{P} = \{p_i = (x_i, y_i) \mid i = 1, 2, \ldots, N_p\} \tag{1}$$

where each loading point p_i is preassigned a fixed cargo quantity q_i. It is further assumed that the dimensions and weights of each cargo unit are identical. The set of delivery points is expressed as

$$\mathcal{D} = \{d_j = (x_j, y_j) \mid j = 1, 2, \ldots, N_d\}. \tag{2}$$

To more realistically reflect the local obstacles and dynamic risks encountered in low-altitude transportation environments, each obstacle is approximated as a circular region. The center coordinates and radius of an obstacle are denoted by

$$o_l = (x_l, y_l), \quad r_l, \quad l = 1, 2, \ldots, N_o \tag{3}$$

which leads to the construction of the obstacle set

$$\mathcal{O} = \{(o_l, r_l) \mid l = 1, 2, \ldots, N_o\}. \tag{4}$$

This representation effectively captures the sparsity of node distribution and the local irregularity of obstacle shapes in practical scenarios.

Within this low-altitude transportation system, drones are the primary agents executing the tasks, and their kinematic characteristics and payload capacities directly affect overall system performance. Let the set of drones be

$$\mathcal{U} = \{u_k \mid k = 1, 2, \ldots, N_u\} \tag{5}$$

with the initial position of each drone at time $t = 0$ given by

$$u_k(0) = \left(x_k^0, y_k^0\right). \tag{6}$$

To capture the dynamic behavior of the drones on the 2-D plane, we adopt a first-order discrete-time dynamic model

$$u_k(t + \Delta t) = u_k(t) + v_k(t)\Delta t \quad \forall t \in [0, T - \Delta t] \tag{7}$$

where $v_k(t)$ is the velocity vector of drone u_k at time t. Moreover, each drone has a fixed payload capacity C, meaning its cargo load $c_k(t)$ satisfies

$$0 \le c_k(t) \le C \quad \forall t. \tag{8}$$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:46:07 UTC from IEEE Xplore. Restrictions apply.

Furthermore, the transportation task is regarded as the cargo transfer process from loading points to delivery points. This process not only involves the motion scheduling of the drones but also requires accuracy and timeliness in both the loading and unloading phases. To ensure this, the system incorporates strict safety constraints. In particular, the distance between any two drones must be no less than the preset safe distance d_{safe} at all times, i.e.,

$$\|u_k(t) - u_{k'}(t)\| \geq d_{\text{safe}} \quad \forall k \neq k' \; \forall t \in [0, T]. \tag{9}$$

Similarly, the distance between any drone and any obstacle must satisfy

$$\begin{aligned} &\|u_k(t) - o_l\| \geq r_l + d_{\text{safe}} \\ &\forall k \in \{1, \ldots, N_u\} \; \forall (o_l, r_l) \in \mathcal{O} \; \forall t \in [0, T] \end{aligned} \tag{10}$$

to ensure effective obstacle avoidance during flight and to reduce the risk of collisions.

B. Problem Description

Based on the above spatial environment and the models for drones and cargo, we now formally describe the scheduling problem for low-altitude transportation tasks. In addition to the constraints of cargo integrity, collision avoidance, dynamics, and payload capacity, the model considers the path cost factors encountered in practical scheduling. Therefore, the optimization objective is not solely to minimize the total scheduling time T but to adopt a comprehensive criterion that reflects both transportation efficiency and operating costs. To this end, the following decision variables are introduced.

1) *Flight Trajectory Variables:* For each drone u_k, its trajectory over the time interval $[0, T]$ is represented by $\{u_k(t)\}_{t \in [0,T]}$.
2) *Task Allocation Variables:* Define the binary variable is

$$x_{ijk} = \begin{cases} 1, & \text{condition} \\ 0, & \text{otherwise} \end{cases} \tag{11}$$

where the condition holds if drone u_k executes the transportation task from loading point p_i to delivery point d_j. And let q_{ij} denote the amount of cargo transported in the task from p_i to d_j. To ensure that the cargo at each loading point is completely allocated, it is typically required that

$$\sum_{k=1}^{N_u} \sum_{j=1}^{N_d} x_{ijk} \, q_{ij} = q_i \quad \forall i = 1, \ldots, N_p. \tag{12}$$

Assume that the total scheduling time required for the system to complete all transportation tasks is T. In order to balance the timeliness of task completion with the flight distance cost, a weighted objective function is adopted

$$\min_{\{u_k(t),\, x_{ijk}\},\, T} \quad \alpha T + \beta \sum_{k=1}^{N_u} \sum_{t=0}^{T-\Delta t} \|u_k(t + \Delta t) - u_k(t)\| \tag{13}$$

where α and β are preset weighting coefficients that balance the minimization of the total scheduling time and the total flight distance, respectively.

In summary, the mathematical model for the low-altitude transportation scheduling system can be formulated as

$$\begin{aligned} \min_{\{u_k(t),\, x_{ijk}\},\, T} \quad & \alpha T + \beta \sum_{k=1}^{N_u} \sum_{t=0}^{T-\Delta t} \|u_k(t + \Delta t) - u_k(t)\| \\ \text{s.t.} \quad & C_1: \quad \sum_{k=1}^{N_u} \Phi_k(T) = \sum_{i=1}^{N_p} q_i \\ & \qquad \Phi_k(T) = \sum_{i=1}^{N_p} \sum_{j=1}^{N_d} x_{ijk} \, q_{ij} \quad \forall k \\ & C_2: \quad \|u_k(t) - u_{k'}(t)\| \geq d_{\text{safe}} \\ & \qquad \forall k \neq k' \; \forall t \in [0, T] \\ & C_3: \quad \|u_k(t) - o_l\| \geq r_l + d_{\text{safe}} \\ & \qquad \forall k, \; \forall (o_l, r_l) \in \mathcal{O} \; \forall t \in [0, T] \\ & C_4: \quad u_k(t + \Delta t) = u_k(t) + v_k(t) \Delta t \\ & \qquad \forall k \; \forall t \in [0, T - \Delta t] \\ & C_5: \quad \sum_{i=1}^{N_p} \sum_{j=1}^{N_d} x_{ijk} \, q_{ij} \leq C \quad \forall k \\ & C_6: \quad x_{ijk} \in \{0, 1\} \quad \forall i = 1, \ldots, N_p \\ & \qquad \forall j = 1, \ldots, N_d \; \forall k = 1, \ldots, N_u \end{aligned} \tag{14}$$

where the objective function employs a weighted approach that integrates both the total scheduling time T and the flight distances of all drones, reflecting a tradeoff between timeliness and operating costs. Constraint C_1 ensures the complete transportation of cargo from each loading point; C_2 and C_3 ensure the safety distances between drones and between drones and obstacles, respectively; C_4 guarantees the kinematic consistency of drone movements; C_5 restricts the payload capacity of each drone; and C_6 ensures the discreteness of the task allocation.

III. Distributed Knowledge-Enhanced Multiagent Reinforcement Learning

In this section, we propose a DKEMARL algorithm to address the cooperative scheduling problem for drones in low-altitude transportation tasks. The algorithm leverages geometric data obtained from a simulation environment to construct heuristic knowledge and combines it with the drones' state information. By employing the MADDPG method under the centralized training with decentralized execution (CTDE) [29] strategy, the method achieves coordinated scheduling among multiple agents. The overall DKEMARL algorithm framework is illustrated in Fig. 3. And the notation used for the DKEMARL algorithm is presented in Table II.

A. Knowledge Embedding and Extended Enhancement

In multiagent cooperative scheduling for low-altitude transportation tasks, the richness of state representation directly affects both the effectiveness of policy learning and flight safety. To enable each agent to comprehensively perceive its own state and environmental risks, we not only normalize the raw state data of each drone but also construct several

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:46:07 UTC from IEEE Xplore. Restrictions apply.

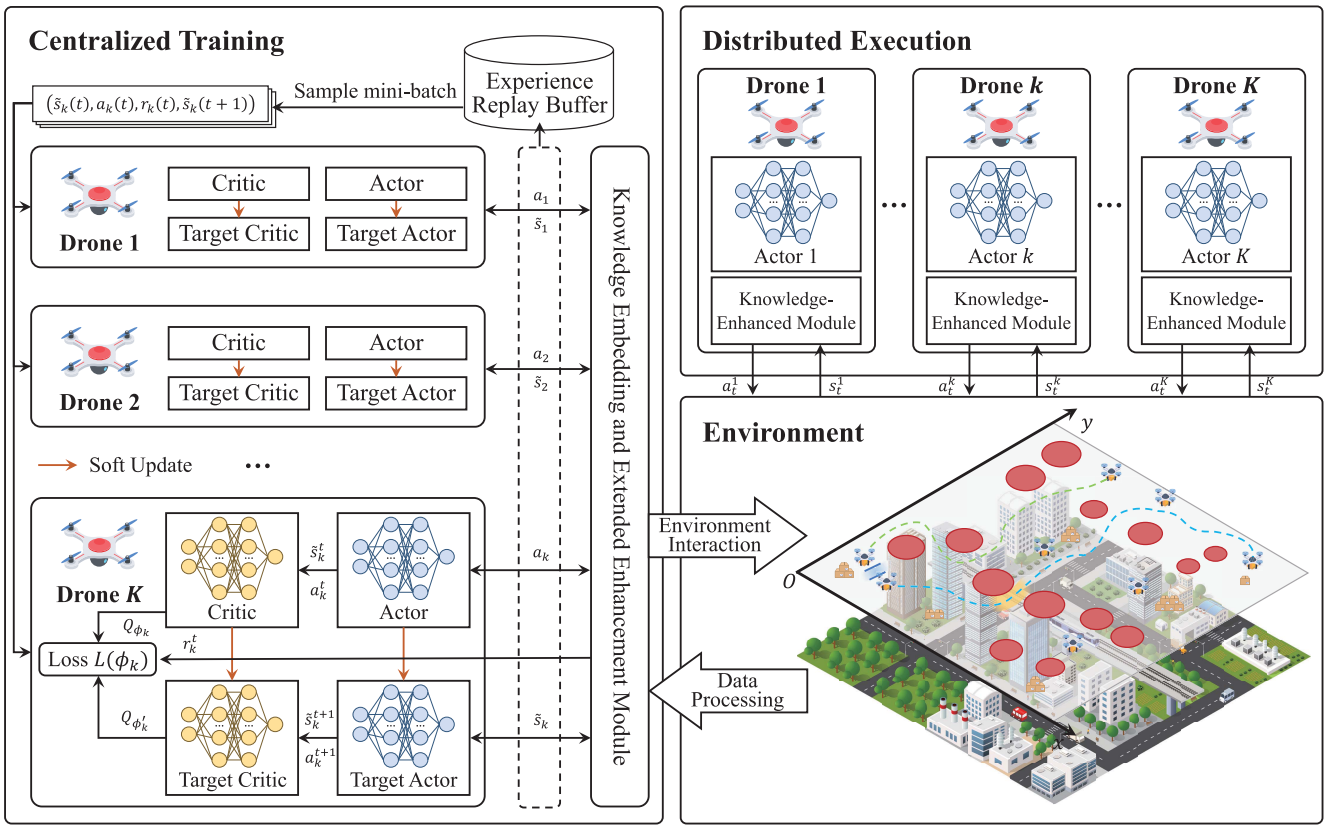

Fig. 3. Architecture of the proposed DKEMARL algorithm encompasses both centralized training and distributed execution stages.

heuristic indicators based on the prior geometric information available in the simulation environment. On this basis, we introduce an estimation of the time-to-collision (TTC) to develop an extended state representation that incorporates both static information and dynamic risk.

Let the raw state vector of a drone u_k at time t be given by

$$s_k(t) = \Big[x_k(t),\ y_k(t),\ v_{x,k}(t),\ v_{y,k}(t),\ c_k(t)\Big] \tag{15}$$

where $c_k(t)$ denotes the current cargo load. To eliminate scale differences among the various physical quantities and facilitate subsequent neural network training, we apply a normalization function $\mathcal{N}(\cdot)$ to the raw state, obtaining the normalized state

$$\bar{s}_k(t) = \mathcal{N}\Big(s_k(t)\Big). \tag{16}$$

Based on the prior geometric information in the environment, we construct several heuristic indicators to provide environmental perception and navigational guidance. Specifically, for each obstacle with center o_l and radius r_l, we first compute the effective distance between drone u_k and obstacle o_l as

$$d^{(l)}_{\text{obs},k}(t) = \|u_k(t) - o_l\| - r_l. \tag{17}$$

Taking the minimum overall obstacles yields the distance from the drone to its nearest obstacle, i.e.,

$$d_{\text{obs},k}(t) = \min_{l=1,\ldots,N_o} d^{(l)}_{\text{obs},k}(t). \tag{18}$$

To facilitate the formulation of safety constraints, this distance is divided by the preset safety distance d_{safe} and capped at 1, resulting in the normalized metric

$$d^{\text{norm}}_{\text{obs},k}(t) = \min\Big\{1,\ \frac{d_{\text{obs},k}(t)}{d_{\text{safe}}}\Big\}. \tag{19}$$

For goal-directed navigation, we compute the Euclidean distance between drone u_k and the target point p_{target}, which can be expressed as

$$d_{\text{target},k}(t) = \|u_k(t) - p_{\text{target}}\| \tag{20}$$

and normalize it as

$$d^{\text{norm}}_{\text{target},k}(t) = \frac{d_{\text{target},k}(t)}{D_{\max}} \tag{21}$$

where $D_{\max}$ denotes the maximum possible distance in the environment. Moreover, the recommended heading for the drone toward the target is computed as

$$\theta_{\text{rec},k}(t) = \text{atan2}\Big(y_{p_{\text{target}}} - y_k(t),\ x_{p_{\text{target}}} - x_k(t)\Big) \tag{22}$$

which provides explicit guidance for the flight path. To further quantify the safety margin between the drone and obstacles, we define a risk score as

$$r_{\text{risk},k}(t) = \exp\Big(-\frac{d_{\text{obs},k}(t)}{d_{\text{safe}}}\Big) \tag{23}$$

so that when the drone is in close proximity to an obstacle, this value approaches 1, indicating a high risk.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:46:07 UTC from IEEE Xplore. Restrictions apply.

TABLE II
LIST OF NOTATIONS FOR DKEMARL

Symbols	Meanings
$s_k(t)$	Raw state
$\bar{s}_k(t)$	Normalized state
$\mathcal{N}(\cdot)$	Normalization function
$d_{\text{obs},k}^{(l)}(t)$	Effective distance to obstacle l
$d_{\text{obs},k}(t)$	Nearest obstacle distance
$d_{\text{obs},k}^{\text{norm}}(t)$	Normalized obstacle distance
$d_{\text{target},k}(t)$	Distance to target
$d_{\text{target},k}^{\text{norm}}(t)$	Normalized target distance
$\theta_{\text{rec},k}(t)$	Recommended heading
$r_{\text{risk},k}(t)$	Risk score
$K_k^{(0)}(t)$	Basic knowledge vector
$TTC_k^{(l)}(t)$	TTC with obstacle l
$TTC_k^{\text{obs}}(t)$	Min TTC (obstacles)
$TTC_{k,k'}(t)$	TTC with drone $u_{k'}$
$TTC_k^{\text{drone}}(t)$	Min TTC (drones)
$TTC_k(t)$	Overall TTC
$TTC_k^{\text{norm}}(t)$	Normalized TTC
$K_k(t)$	Extended knowledge vector
$\tilde{s}_k(t)$	Final state
ω_s, ω_K	Fusion weights
$v_k(t)$	Velocity vector
ϵ	Small constant
$\mathbf{I}\{\cdot\}$	Indicator function
β	Safety penalty weight
$r_{\text{env}}(t)$	Environmental reward
$r_k(t)$	Total reward
$\pi_{\theta_k}(a\|\tilde{s}_k(t))$	Policy function
$Q_{\phi_k}(\cdot)$	Joint Q-function
$L(\phi_k)$	Critic loss
y	Target TD value
γ	Discount factor
a'_k	Target action
τ	Soft update factor
ϕ'_k, θ'_k	Target network parameters

By combining these indicators, we construct a basic knowledge vector

$$K_k^{(0)}(t) = \Big[d_{\text{obs},k}(t),\ d_{\text{target},k}(t),\ \theta_{\text{rec},k}(t),\ r_{\text{risk},k}(t)\Big]. \quad (24)$$

To further enhance the state representation with respect to dynamic risk, we introduce an estimation of the TTC. This indicator provides an early measure of potential collision risk by considering the drone's current motion state relative to obstacles or other drones.

For collision risk with obstacles, consider the relationship between drone u_k and obstacle o_l. Utilizing the previously computed effective distance and the drone's speed (with a small constant $\epsilon > 0$ added to avoid division by zero), the TTC is defined as

$$\text{TTC}_k^{(l)}(t) = \frac{d_{\text{obs},k}^{(l)}(t)}{\|v_k(t)\| + \epsilon}. \quad (25)$$

Taking the minimum overall obstacles yields

$$\text{TTC}_k^{\text{obs}}(t) = \min_{l=1,\ldots,N_o} \text{TTC}_k^{(l)}(t) \quad (26)$$

which serves as a measure of the collision risk between the drone and obstacles.

In a multidrone cooperative scenario, to prevent interagent collisions, for any two distinct drones u_k and $u_{k'}$, we first compute the effective distance (i.e., the distance minus the safety distance) and then, considering the relative speed, define the TTC as

$$\text{TTC}_{k,k'}(t) = \frac{\|u_k(t) - u_{k'}(t)\| - d_{\text{safe}}}{\|v_k(t) - v_{k'}(t)\| + \epsilon}. \quad (27)$$

The minimum value over all other drones is then given by

$$\text{TTC}_k^{\text{drone}}(t) = \min_{k' \neq k} \text{TTC}_{k,k'}(t). \quad (28)$$

By integrating both the obstacle-related and interagent collision risks, the overall TTC for drone u_k is defined as

$$\text{TTC}_k(t) = \min\Big\{\text{TTC}_k^{\text{obs}}(t),\ \text{TTC}_k^{\text{drone}}(t)\Big\}. \quad (29)$$

To maintain consistency with the other indicators, we normalize $\text{TTC}_k(t)$ by setting a maximum possible value $\text{TTC}_{\max}$, resulting in

$$\text{TTC}_k^{\text{norm}}(t) = \min\left\{1,\ \frac{\text{TTC}_k(t)}{\text{TTC}_{\max}}\right\}. \quad (30)$$

The normalized TTC indicator is then integrated into the basic knowledge vector to form the extended knowledge vector

$$K_k(t) = \Big[d_{\text{obs},k}(t),\ d_{\text{target},k}(t),\ \theta_{\text{rec},k}(t),\ r_{\text{risk},k}(t),\ \text{TTC}_k(t)\Big]. \quad (31)$$

Finally, by fusing the normalized state $\bar{s}_k(t)$ and the extended knowledge vector $K_k(t)$ using predetermined weights, we construct the final extended state representation

$$\tilde{s}_k(t) = \omega_s\, \bar{s}_k(t) + \omega_K\, K_k(t) \quad (32)$$

where ω_s and ω_K are hyperparameters determined via simulation experiments. A multilayer perceptron (MLP) is then applied to the fused state for deep feature extraction, yielding a more compact state representation with improved discriminative capability.

In summary, the extended state $\tilde{s}_k(t)$ not only captures the static positional, velocity, and cargo information of the drone but also incorporates dynamic risk information related to obstacles and other drones. This comprehensive state representation provides a more accurate and objective basis for decision-making in the MADDPG-based multiagent cooperative scheduling algorithm, facilitating a balanced consideration of task timeliness and overall system safety.

B. Reward Design and Safety Constraint Mechanism

To ensure that drones not only accomplish transportation tasks efficiently but also adhere strictly to flight safety requirements, the proposed algorithm establishes a comprehensive reward mechanism that explicitly addresses both task performance and safety considerations. The reward received by each drone at every time step reflects its success in executing the mission as well as its compliance with safety protocols.

For task performance, the reward is determined by fixed, predetermined values. When a drone successfully completes the loading, transportation, and delivery of cargo, it is awarded a fixed positive reward, which explicitly incentivizes timely and accurate task completion. If the drone fails to complete the task within the prescribed time, it incurs a fixed negative reward, thereby penalizing delays. In addition, if the drone takes a discernible detour that results in a deviation from the

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:46:07 UTC from IEEE Xplore. Restrictions apply.

shortest path, a fixed negative reward is imposed to encourage the selection of a more direct and effective flight path. This design quantifies task performance and provides clear positive feedback during the learning process.

Regarding flight safety, the algorithm monitors the distances between a drone and other drones as well as obstacles and enforces a safety constraint penalty mechanism. A predetermined safety distance, d_{safe}, is defined. If the distance between drone u_k and any other drone or the nearest obstacle is less than d_{safe}, a safety penalty is triggered. Specifically, the safety penalty is defined as

$$L_{\text{safe},k}(t) = \sum_{k' \neq k} \mathbf{I}\Big\{\|u_k(t) - u_{k'}(t)\| < d_{\text{safe}}\Big\} + \mathbf{I}\Big\{d_{\text{obs},k}(t) < d_{\text{safe}}\Big\} \tag{33}$$

where $\mathbf{I}\{\cdot\}$ is an indicator function that takes the value 1 when the condition is met and 0 otherwise. This formulation ensures that if the distance between the drone and another drone or obstacle falls below d_{safe}, an immediate penalty signal is generated to enforce safe flight behavior. The weight of the safety penalty, β, is determined through extensive simulation experiments to a fixed value, ensuring that the penalty's influence on the overall reward is sufficient to deter unsafe behavior arising from efforts to maximize task efficiency.

In summary, the total reward for each drone at time t is defined as

$$r_k(t) = r_{\text{env}}(t) - \beta\, L_{\text{safe},k}(t). \tag{34}$$

This reward mechanism guarantees that even if a drone may achieve a high task completion reward through certain strategies, any unsafe behavior will immediately incur a penalty that reduces the overall reward. Consequently, the reinforcement learning algorithm is compelled to balance task performance with strict adherence to flight safety. Through this comprehensive and explicit reward design, each drone receives clear positive feedback during training, ultimately leading to an efficient and safe cooperative scheduling strategy.

C. Design of the DKEMARL Algorithm

The DKEMARL algorithm employs centralized training with a decentralized execution strategy that integrates global information utilization during the training phase with local decision-making during execution based on the MADDPG method, as shown in Algorithm 1. In the training phase, each drone uses its extended state $\tilde{s}_k(t)$ as input to generate an action selection policy $\pi_{\theta_k}(a|\tilde{s}_k(t))$ via its individual policy network (actor). A centralized value network (critic) is introduced during training to evaluate the interactions among multiple agents accurately. This critic not only utilizes the local observations of each agent but also incorporates information from other agents, thereby computing the joint action-value function $Q_{\phi_k}(s, a_1, \ldots, a_K)$.

During training, the experience replay buffer stores samples collected by each drone in the form $(\tilde{s}_k(t),\ a_k(t),\ r_k(t),\ \tilde{s}_k(t+1))$. These samples are then used to train the critic network

Algorithm 1 Pseudocode of DKEMARL

Input: State s, environmental info, safety distance d_{safe}, fusion weights ω_s, ω_K, hyperparameters γ, β, τ
Output: Joint policy π and global Q-function Q

```
Initialize each drone's actor π_i, critic Q_i and target networks π'_i, Q'_i, replay buffer D
for each episode do
    Reset environment, obtain initial state s(0)
    for each time step t do
        for each drone i do
            State Enhancement: Normalize s_i,
            compute heuristic K_i, obtain s̃_i ← ω_s s_i + ω_K K_i
            Action Selection: a_i ← π_i(s̃_i)
            Execute action, observe environment reward r_i^env
            and next state s'_i
            Compute safety penalty L_safe,i
            Update reward: r_i ← r_i^env − β L_safe,i
            Store experience (s̃_i, a_i, r_i, s̃'_i) in D
        end for
    end for
    Training Update:
    Sample a mini-batch from D
    for each drone i do
        Compute target action: a'_i ← π'_i(s̃'_i)
        Compute target Q-value:
        y_i ← r_i + γ Q'_i(s̃', a'_1, ..., a'_N)
        Update critic Q_i (minimize error)
        Update actor π_i (via policy gradient)
        Soft update target networks:
        π'_i ← τ π_i + (1 − τ) π'_i
        Q'_i ← τ Q_i + (1 − τ) Q'_i
    end for
end for
```

with the objective of minimizing the temporal-difference (TD) error. Specifically, the loss function is defined as

$$L(\phi_k) = \mathbb{E}_{(s,a,r,s')\sim D}\Big[\Big(Q_{\phi_k}(s, a_1, \ldots, a_K) - y\Big)^2\Big] \tag{35}$$

where the target value is computed by

$$y = r_k(t) + \gamma\, Q_{\phi'_k}\Big(s', a'_1, \ldots, a'_K\Big) \tag{36}$$

with the target network $Q_{\phi'_k}$ providing the necessary target output. The target action is obtained by

$$a'_k = \pi_{\theta'_k}(\tilde{s}_k(t+1)) \tag{37}$$

and the target network parameters are updated using a soft update strategy

$$\phi'_k \leftarrow \tau\, \phi_k + (1-\tau)\, \phi'_k, \quad \theta'_k \leftarrow \tau\, \theta_k + (1-\tau)\, \theta'_k \tag{38}$$

where $\tau \ll 1$ is the soft update factor ensuring that the target network smoothly tracks the current network.

After the critic network has been updated, the actor network is optimized using policy gradient methods. The gradient

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:46:07 UTC from IEEE Xplore. Restrictions apply.

estimate is given by

$$\nabla_{\theta_k} J \approx \mathbb{E}_{\tilde{s}_k \sim D}\Big[\nabla_a Q_{\phi_k}(s, a_1, \ldots, a_K)\big|_{a_k=\pi_{\theta_k}(\tilde{s}_k)} \cdot \nabla_{\theta_k} \pi_{\theta_k}(\tilde{s}_k)\Big]. \quad (39)$$

This procedure enables the actor network to generate actions that are evaluated as having higher value by the critic network, thereby continuously enhancing the overall cooperative scheduling performance.

In summary, during the training phase, a centralized parameter update strategy is employed to promote consistency among the agents' models. In contrast, during the execution phase, each drone relies solely on its local extended state for decision-making, which reduces communication overhead and ensures that the drone swarm can make decisions in real time, efficiently, and robustly.

IV. Simulation Results

This section presents a detailed description of the simulation experiments designed to validate the applicability of the IoD task scheduling method in low-altitude transportation scenarios. These experiments provide a comprehensive evaluation of task scheduling efficiency, flight path optimization, and the drones' obstacle avoidance capabilities, thereby ensuring that the proposed method effectively addresses real-world challenges.

The primary objectives of the experiments are to verify several key aspects: task scheduling efficiency, the optimization of flight path planning, and the drones' obstacle avoidance capabilities. By comparing the performance of various scheduling algorithms, we aim to reduce task completion times and enhance flight path safety.

A. Simulation Environment

The simulation environment is constructed using the OpenAI Gym [30] framework, which provides a standardized and flexible platform for developing and testing reinforcement learning algorithms. The entire simulation is set in a 2-D Euclidean space $\mathbb{R}^2$, representing a simplified low-altitude transportation scenario. The space is defined within a rectangular area of size $L_x \times L_y$, where $L_x = 200$ m and $L_y = 200$ m. This size is chosen to mimic a real-world low-altitude area where drones can operate, providing enough space for multiple drones to perform tasks without overcrowding.

The set of loading points $\mathcal{P} = \{p_i = (x_i, y_i) \mid i = 1, 2, \ldots, N_p\}$ is randomly generated within the left-hand side of the simulation area. The number of loading points N_p is set to 8. Each loading point p_i is assigned a fixed amount of cargo q_i, which is uniformly distributed in the range [10, 30] units. For simplicity, the cargo units are assumed to be of the same size and weight. The coordinates of the loading points are generated using a uniform distribution within the subarea $[0, L_x/2] \times [0, L_y]$.

Similarly, the set of delivery points $\mathcal{D} = \{d_j = (x_j, y_j) \mid j = 1, 2, \ldots, N_d\}$ is randomly placed in the simulation area. The number of delivery points N_d is also set to 8. Similar to the loading points, the coordinates of the delivery points are generated using a uniform distribution within the subarea $[L_x/2, L_x] \times [0, L_y]$. This separation of loading and delivery points mimics a typical transportation scenario where goods are picked up from one area and delivered to another.

The obstacles in the environment are approximated as circular regions. The set of obstacles $\mathcal{O} = \{(o_l, r_l) \mid l = 1, 2, \ldots, N_o\}$, where $o_l = (x_l, y_l)$ is the center coordinate of the lth obstacle and r_l is its radius. The number of obstacles N_o is set in the range [5, 15]. The center coordinates o_l are randomly generated within the entire simulation area, and the radii r_l are uniformly distributed in the range [5, 20] m. This range of radii is chosen to represent different sizes of obstacles that drones may encounter in a low-altitude environment, such as buildings or no-fly zones.

Finally, The set of drones $\mathcal{U} = \{u_k \mid k = 1, 2, \ldots, N_u\}$, with $N_u \in [3, 12]$ drones. Each drone u_k has an initial position $u_k(0) = (x_k^0, y_k^0)$ at time $t = 0$. The initial positions of the drones are uniformly distributed in a small starting area located near the center of the left-hand side of the simulation area. Each drone follows a discrete-time first-order dynamics model $u_k(t + \Delta t) = u_k(t) + v_k(t)\Delta t$, where $\Delta t = 0.2$ s is the time step. The speed vector $v_k(t)$ of each drone is bounded, with a maximum speed limit of $v_{\max} = 5$ meters per second. Each drone has a fixed cargo capacity $C \in [0, 5]$ units and its cargo load $c_k(t)$ at time t satisfies $0 \le c_k(t) \le C$. In addition, two important safety constraints are implemented in the simulation. First, the minimum safe distance between any two drones is set to $d_{safe} = 2$ m. That is, $||u_k(t) - u_{k'}(t)|| \ge d_{safe}$ for all $k \ne k'$ and $t \in [0, T]$. Second, the distance between each drone and any obstacle must satisfy $||u_k(t) - o_l|| \le r_l + d_{safe}$ for all $k \in \{1, \ldots, N_u\}$, $(o_l, r_l) \in O$, and $t \in [0, T]$. These safety constraints ensure that the drones can operate safely in the environment without colliding with each other or with obstacles.

B. Compared Algorithm Settings

To assess the efficacy of the proposed DKEMARL algorithm, a comparative analysis is conducted against the standard MADDPG algorithm within the simulation environment described earlier. Both algorithms employ identical network architectures for their actor and critic components, consisting of multilayer perceptrons with two hidden layers of 128 units each, activated by ReLU functions, and a tanh activation in the actor's output layer. The hyperparameters are also uniformly configured to ensure fairness: the actor learning rate is set to 10^{-3}, the critic learning rate to 10^{-2}, the discount factor γ to 0.99, the soft update parameter τ to 0.01, the replay buffer size to 10^6, and the batch size to 128. The primary distinctions between DKEMARL and MADDPG lie in their state representations and reward mechanisms, which are elaborated in the following paragraphs.

The DKEMARL algorithm enhances the raw state information through a knowledge embedding and extension module that incorporates heuristic indicators derived from prior environmental knowledge. Its state representation includes normalized drone positions, velocities, and cargo loads, enriched with additional features such as the distance

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:46:07 UTC from IEEE Xplore. Restrictions apply.

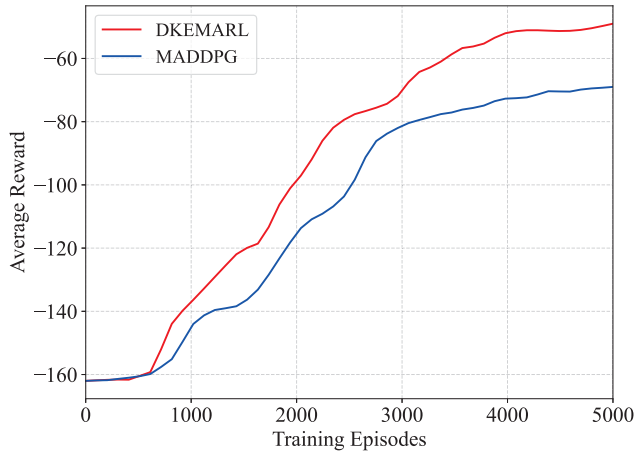

Fig. 4. Average reward over training episodes for diverse algorithms.

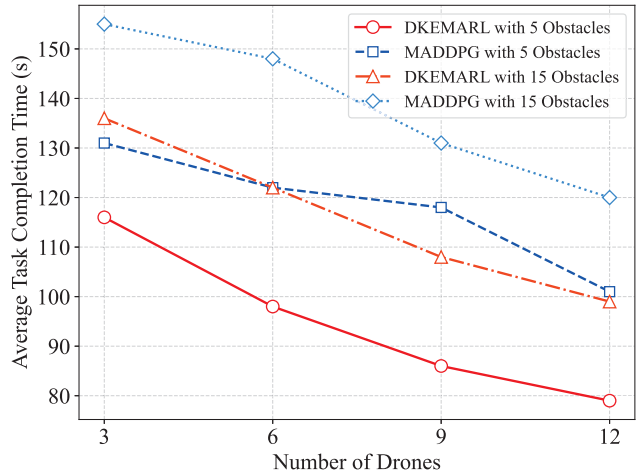

Fig. 5. Average task completion time with varying numbers of drones.

to the nearest obstacle $d_{\text{obs},k}(t)$, the distance to the target $d_{\text{target},k}(t)$, the recommended heading $\theta_{\text{rec},k}(t)$, the risk score $r_{\text{risk},k}(t)$, and the estimated TTC $\text{TTC}_k(t)$, fused with the raw state using weights $\omega_s = 0.5$ and $\omega_K = 0.5$ for balanced integration. The reward mechanism is carefully crafted to balance task timeliness and flight safety, offering positive rewards for successful task completion, such as loading, transporting, and delivering cargo, while imposing negative rewards for delays or detours. Additionally, a safety penalty $L_{\text{safe},k}(t)$ is applied whenever the distance between drones or between a drone and an obstacle falls below the predefined safety threshold d_{safe}, emphasizing safety compliance throughout the operation.

In contrast, the MADDPG algorithm relies solely on a state representation comprising the normalized raw state information, including the drone's position, velocity, and cargo load, processed with the same normalization function $\mathcal{N}(\cdot)$ as DKEMARL, but without any supplementary heuristic indicators or knowledge enhancement modules. Its reward mechanism mirrors DKEMARL by providing positive rewards for task completion and negative rewards for delays or detours; however, it lacks an explicit safety penalty, depending instead on the simulation environment to manage safety violations through measures like episode termination or indirect penalties. As a result, MADDPG places less emphasis on enforcing stringent safety constraints compared to DKEMARL, highlighting a key difference in their approach to handling the complexities of low-altitude drone transportation tasks.

C. Performance Comparison of Algorithms

This section compares the performance of the DKEMARL algorithm with the MADDPG algorithm in coordinating drone swarms for urban logistics tasks. The evaluation draws on simulation results, focusing on key performance aspects: learning efficiency, scalability with varying drone counts, robustness to obstacle density, and safety compliance.

As shown in Fig. 4, which plots the average reward over 5000 training episodes for both DKEMARL and MADDPG, illustrates the learning efficiency of both algorithms as they refine their scheduling strategies. Initially, both DKEMARL and MADDPG exhibit comparable performance, starting with lower rewards that indicate suboptimal policies. As training progresses, however, a clear divergence emerges: DKEMARL consistently achieves higher rewards compared to MADDPG, with the gap widening in later episodes. This trend suggests that DKEMARL learns more effectively, optimizing the balance between task completion and safety more successfully. The superior performance can be attributed to DKEMARL's use of prior environmental knowledge, such as obstacle locations and target directions, which enriches its decision-making process beyond the raw state data utilized by MADDPG.

Fig. 5 comparing average task completion time against the number of drones reveals how both algorithms scale with swarm size under different obstacle densities. As the number of drones increases, task completion time decreases for both DKEMARL and MADDPG, reflecting the benefit of distributing tasks across more agents. However, DKEMARL consistently demonstrates a steeper reduction in completion time compared to MADDPG, particularly in scenarios with higher obstacle densities. This indicates that DKEMARL is more efficient at coordinating larger swarms and navigating complex environments. The advantage likely stems from its knowledge-enhanced approach, which incorporates heuristic indicators like obstacle proximity, enabling better path planning and task allocation in cluttered urban settings.

Fig. 6 examines the robustness of the algorithms as the number of obstacles increases, testing their performance with different swarm sizes. For both DKEMARL and MADDPG, task completion time rises with more obstacles due to the added complexity of path planning and avoidance. Nevertheless, DKEMARL exhibits a less pronounced increase in completion time compared to MADDPG across all swarm sizes. This trend highlights DKEMARL's greater adaptability to environmental challenges, maintaining efficiency even as urban complexity grows. The inclusion of dynamic risk indicators, such as TTC, in DKEMARL's state representation likely contributes to this resilience, allowing it to anticipate and mitigate obstacles more effectively than MADDPG.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:46:07 UTC from IEEE Xplore. Restrictions apply.

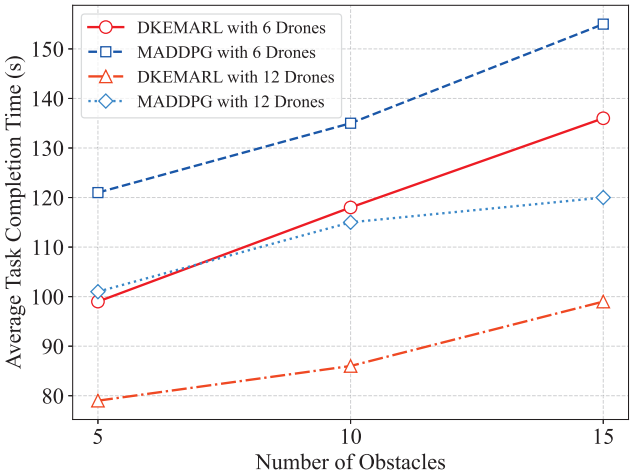

Fig. 6. Average task completion time with varying numbers of obstacles.

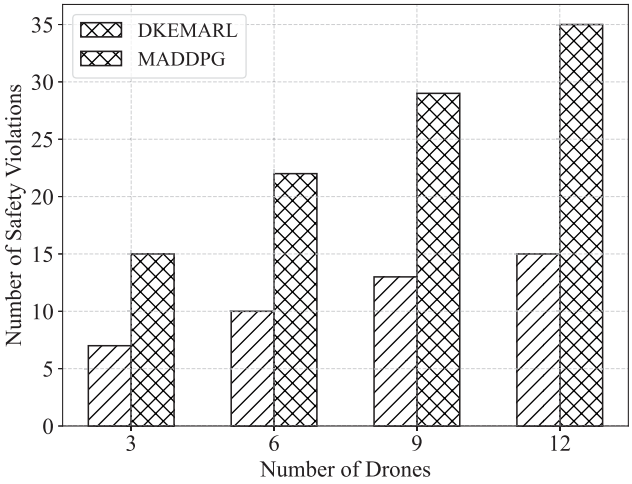

Fig. 7. Number of safety violations with varying numbers of drones.

As depicted in Fig. 7, the safety performance figure tracks the number of safety violations, defined as instances where drones breach minimum safety distances, as the number of drones increases. Both algorithms show an upward trend in violations with larger swarms, as denser drone populations heighten the risk of proximity issues. However, DKEMARL consistently records fewer violations than MADDPG across all drone counts. This disparity underscores DKEMARL's stronger emphasis on safety, achieved through an explicit penalty mechanism in its reward function that discourages unsafe behaviors. In contrast, MADDPG's reliance on indirect environmental feedback appears less effective at enforcing safety constraints, making DKEMARL a safer choice for urban operations where collision avoidance is critical.

The experimental results collectively demonstrate DKEMARL's superiority over MADDPG in managing IoD swarms for urban logistics. In terms of learning efficiency, DKEMARL achieves higher average rewards, indicating a more effective optimization of scheduling strategies that balance timeliness and safety. Its scalability is evident in the reduced task completion times with increasing drone numbers, particularly in obstacle-rich environments, showcasing its ability to coordinate larger swarms efficiently. DKEMARL also exhibits greater robustness, with less performance degradation as obstacle density rises, reflecting its adaptability to complex urban conditions. Finally, its safety performance stands out, with fewer violations due to a proactive safety-focused design. These advantages arise from DKEMARL's innovative integration of environmental knowledge into the reinforcement learning framework, enhancing both state representation and reward mechanisms.

V. Conclusion

This article introduces DKEMARL, an advanced algorithm engineered to enhance coordination in IoD deployments for urban logistics amid complex environments marked by high-rise buildings and restricted airspace. DKEMARL integrates prior environmental knowledge via a novel Knowledge Embedding and Expansion module and builds on the MADDPG framework under a centralized-training, decentralized-execution scheme. This combination delivers an optimal tradeoff between mission timeliness and flight safety. Looking ahead, we plan to incorporate real-time factors, such as weather variations and evolving airspace restrictions, and to extend our approach to larger-scale drone swarms, thereby laying the groundwork for a resilient, distributed, and intelligent urban transportation network.

References

[1] L. Abualigah, A. Diabat, P. Sumari, and A. H. Gandomi, "Applications, deployments, and integration of Internet of Drones (IoD): A review," *IEEE Sensors J.*, vol. 21, no. 22, pp. 25532–25546, Nov. 2021.

[2] E. N. Barmpounakis, E. I. Vlahogianni, and J. C. Golias, "Unmanned aerial aircraft systems for transportation engineering: Current practice and future challenges," *Int. J. Transp. Sci. Technol.*, vol. 5, no. 3, pp. 111–122, Oct. 2016.

[3] G. Zhao, Y. Wang, T. Mu, Z. Meng, and Z. Wang, "Reinforcement-learning-assisted multi-UAV task allocation and path planning for IIoT," *IEEE Internet Things J.*, vol. 11, no. 16, pp. 26766–26777, Aug. 2024.

[4] J. Li, Y. Xiong, and J. She, "UAV path planning for target coverage task in dynamic environment," *IEEE Internet Things J.*, vol. 10, no. 20, pp. 17734–17745, Oct. 2023.

[5] Z. Fang, S. Hu, J. Wang, Y. Deng, X. Chen, and Y. Fang, "Prioritized information bottleneck theoretic framework with distributed online learning for edge video analytics," *IEEE Trans. Netw.*, vol. 33, no. 3, pp. 1203–1219, Jun. 2025.

[6] Z. Fang et al., "PACP: Priority-aware collaborative perception for connected and autonomous vehicles," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 15003–15018, Dec. 2024.

[7] J. Huang, B. Wu, Q. Duan, L. Dong, and S. Yu, "A fast UAV trajectory planning framework in RIS-assisted communication systems with accelerated learning via multithreading and federating," *IEEE Trans. Mobile Comput.*, early access, Feb. 24, 2025, doi: 10.1109/TMC.2025.3544903.

[8] J. Wang, Z. Jiao, J. Chen, X. Hou, T. Yang, and D. Lan, "Blockchain-aided secure access control for UAV computing networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 6, pp. 5267–5279, Nov./Dec. 2024.

[9] J. Wang, J. Wang, Z. Tong, Z. Jiao, M. Zhang, and C. Jiang, "ACBFT: Adaptive chained Byzantine fault-tolerant consensus protocol for UAV ad hoc networks," *IEEE Trans. Veh. Technol.*, early access, Mar. 25, 2025, doi: 10.1109/TVT.2025.3548281.

[10] H. Huang, J. Su, and F.-Y. Wang, "The potential of low-altitude airspace: The future of urban air transportation," *IEEE Trans. Intell. Veh.*, vol. 9, no. 8, pp. 5250–5254, Aug. 2024.

[11] J. Chen, M. Li, Z. Yuan, and Q. Gu, "An improved A* algorithm for UAV path planning problems," in *Proc. IEEE 4th Inf. Technol., Netw., Electron. Autom. Control Conf.*, 2020, pp. 958–962.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:46:07 UTC from IEEE Xplore. Restrictions apply.

[12] E. Dhulkefl, A. Durdu, and H. Terzioğlu, "Dijkstra algorithm using UAV path planning," *Konya J. Eng. Sci.*, vol. 8, pp. 92–105, Dec. 2020.

[13] Y. Xia, W. Zeng, X. Xing, Y. Zhan, K. H. Tan, and A. Kumar, "Joint optimisation of drone routing and battery wear for sustainable supply chain development: A mixed-integer programming model based on blockchain-enabled fleet sharing," *Ann. Oper. Res.*, vol. 327, pp. 89–127, Dec. 2023.

[14] M. Radmanesh and M. Kumar, "Flight formation of UAVs in presence of moving obstacles using fast-dynamic mixed integer linear programming," *Aerosp. Sci. Technol.*, vol. 50, pp. 149–160, Mar. 2016.

[15] G. Wang, F. Wang, J. Wang, M. Li, L. Gai, and D. Xu, "Collaborative target assignment problem for large-scale UAV swarm based on two-stage greedy auction algorithm," *Aerosp. Sci. Technol.*, vol. 149, Jun. 2024, Art. no. 109146.

[16] M. Dorigo, M. Birattari, and T. Stutzle, "Ant colony optimization," *IEEE Comput. Intell. Mag.*, vol. 1, no. 4, pp. 28–39, Nov. 2007.

[17] D. Wang, D. Tan, and L. Liu, "Particle swarm optimization algorithm: An overview," *Soft Comput.*, vol. 22, no. 2, pp. 387–408, Jan. 2017.

[18] J. H. Holland, "Genetic algorithms," *Sci. Amer.*, vol. 267, no. 1, pp. 66–73, Jul. 1992.

[19] T. Bäck and H.-P. Schwefel, "An overview of evolutionary algorithms for parameter optimization," *Evol. Comput.*, vol. 1, no. 1, pp. 1–23, Mar. 1993.

[20] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. 31st Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–12.

[21] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 1–13.

[22] C. Yu et al., "The surprising effectiveness of PPO in cooperative multi-agent games," in *Proc. 36th Adv. Neural Inf. Process. Syst.*, 2022, pp. 24611–24624.

[23] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 2961–2970.

[24] J. Su, S. Adams, and P. Beling, "Value-decomposition multi-agent actor-critics," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 11352–11360.

[25] T. Rashid, M. Samvelyan, C. Schroeder, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4295–4304.

[26] J. Wang, Z. Ren, T. Liu, Y. Yu, and C. Zhang, "QPLEX: Duplex dueling multi-agent Q-learning," in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 1–27.

[27] T. Zhang et al., "Multi-agent collaboration via reward attribution decomposition," in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 1–19.

[28] H. Fu, J. Wang, J. Chen, P. Ren, Z. Zhang, and G. Zhao, "Dense multiagent reinforcement learning aided multi-UAV information coverage for vehicular networks," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21274–21286, Jun. 2024.

[29] F. A. Oliehoek, M. T. J. Spaan, and N. Vlassis, "Optimal and approximate Q-value functions for decentralized POMDPs," *J. Artif. Intell. Res.*, vol. 32, no. 1, pp. 289–353, May 2008.

[30] G. Brockman et al., "OpenAI gym," 2016, *arXiv:1606.01540*.

Ye Wang (Student Member, IEEE) received the B.S. degree in computer science and technology from Henan Normal University, Xinxiang, China, in 2022, and the M.S. degree in software engineering from Zhengzhou University, Zhengzhou, China, in 2025. He is currently pursuing the Ph.D. degree with the School of Cyber Science and Technology from Beihang University, Beijing, China, and Zhengzhou University.

His research interests lie in reinforcement learning, deep learning, and swarm intelligence.

Jingjing Wang (Senior Member, IEEE) received the B.Sc. (with Highest Hons.) degree in electronic information engineering from Dalian University of Technology, Dalian, Liaoning, China, in 2014, and the Ph.D. (with Highest Hons.) degree in information and communication engineering from Tsinghua University, Beijing, China, in 2019.

He is currently a Professor with the School of Cyber Science and Technology, Beihang University, Beijing, and also a Researcher with Hangzhou Innovation Institute, Beihang University, Hangzhou, China. His research interests lie in AI-enhanced next-generation wireless networks and swarm intelligence.

Ruijie Zhu (Member, IEEE) received the Ph.D. degree from the State Key Laboratory of Information Photonics and Optical Communication, Beijing, China, in 2017.

He was a Visiting Student with the University of Texas at Dallas, Richardson, TX, USA, under the supervision of Prof. Jason P. Jue. He is currently a Professor with the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing, and also a Researcher with Zhengzhou University, Zhengzhou, China. His research interests lie in multiagent systems, satellite optical networks, and network virtualization.

Ziyang Wang (Member, IEEE) received the Ph.D. degree in electromagnetic wave and microwave technology from Xidian University, Xi'an, China, in 2018.

He was previously a Postdoctoral Fellow with Tsinghua University, Beijing. He is currently an Associate Professor with the School of Electronic and Information Engineering, Beihang University, Beijing. His research interests lie in intelligent metasurfaces, global optimization algorithms, and 5G MIMO array antenna decoupling.

Pujie Xin (Member, IEEE) received the B.S. degree in electrical engineering and automation from the University of Electronic Science and Technology of China, Chengdu, China, in 2018, the M.S. degree in industrial engineering from Rutgers University, New Brunswick, NJ, USA, in 2019, and the Ph.D. degree in mechanical engineering from Temple University, Philadelphia, PA, USA, in 2024.

He is currently an Assistant Professor with the School of Communication Engineering, Hangzhou Dianzi University, Hangzhou, China. His research interests lie in multirobot systems and swarm intelligence.

Pen Pan (Member, IEEE) received the B.Eng. and Ph.D. degrees in electronic engineering from Beihang University, Beijing, China, in 2005 and 2011, respectively.

From December 2007 to December 2008, he was a Visiting Ph.D. Student with the School of Electronics and Computer Science, The University of Southampton, Southampton, U.K. He is currently a Professor with the School of Communication Engineering, Hangzhou Dianzi University, Hangzhou, China. His research interests lie in MIMO systems, signal processing for LEO, and UAV-based communications.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 08,2026 at 04:46:07 UTC from IEEE Xplore. Restrictions apply.